

SEGUNDA ENTREGA PROYECTO - COMPUTACIÓN VISUAL

Presentado por:

Juan José Medina Guerrero - jmedinagu@unal.edu.co
Cristian Daniel Montañez Pineda - cmontanez@unal.edu.co
Sergio Alejandro Ruiz Hurtado - seruizh@unal.edu.co
Daniel Felipe Soracipa - dsoracipa@unal.edu.co

Profesor:

Arbey Aragon Bohorquez

Junio 18 de 2025



Universidad Nacional de Colombia
Facultad de Ingeniería
Departamento de Ingeniería de
Sistemas e Industrial
2025

CONTENIDO:

SEGUNDA ENTREGA PROYECTO - COMPUTACIÓN VISUAL.....	0
Storytelling del Proyecto.....	2
Levantamiento de Requerimientos.....	3
Requerimientos Técnicos y Tecnológicos.....	5
Protocolos de Comunicación.....	5
Diagramas y Arquitectura.....	7
1. Diagrama general del sistema.....	7
2. Diagrama de interacción de usuario.....	8
3. Módulos previstos.....	8
4. Principales dependencias y riesgos técnicos.....	9
Repositorios y Recursos.....	10
Recursos de Soporte y Referencia.....	10
Mockups o Prototipos.....	11
Vista Principal – Dashboard Interactivo.....	12
Módulo 1: Detección en Tiempo Real (Cámara).....	12
Módulo 2: Análisis de Imagen Estática.....	12
Módulo 3: Asistente de Reciclaje.....	13
Módulo 4: Visualizador 3D.....	13

Storytelling del Proyecto

Problema

En las ciudades modernas, muchas personas desconocen cómo reciclar correctamente. Aunque separan sus residuos, no siempre comprenden a qué categoría pertenecen o qué acciones deben tomar antes de desecharlos. Esta falta de conocimiento práctico reduce la efectividad de los programas de reciclaje.

Solución Propuesta

Nuestro proyecto busca educar al usuario mientras interactúa con sus propios residuos, permitiéndole cargar imágenes o usar su cámara para detectar objetos reciclables. Luego, a través de modelos de inteligencia artificial como YOLO, identificamos estos elementos y mostramos visualizaciones 3D representativas. Finalmente, utilizando ChatGPT, generamos consejos personalizados para reciclar de forma adecuada.

La experiencia completa ocurre en un entorno web accesible, con visualización interactiva en Three.js que guía al usuario de forma dinámica y educativa.

Escenarios de Uso y Flujo del Usuario

Escenario 1: Imagen Estática

- El usuario entra a la web y lee sobre el reciclaje.
- Carga una foto de sus residuos (por ejemplo, una mesa con botellas y papel).
- El sistema detecta los objetos y muestra una escena 3D interactiva con esos ítems representados visualmente.
- Al hacer clic en cada objeto, se muestra un consejo personalizado para su reciclaje.

Escenario 2: Detección en Tiempo Real

- El usuario activa su cámara.
- El sistema detecta en tiempo real objetos reciclables.
- Cada objeto se visualiza con color y forma 3D.
- El sistema responde con sugerencias mientras se actualiza dinámicamente la escena.

Objetivos del Sistema

Visuales:

- Crear una experiencia 3D atractiva y fluida con Three.js.
- Renderizar modelos 3D reconocibles de los objetos de basura comunes.
- Ofrecer una interfaz de usuario intuitiva y estéticamente agradable.

Educativos:

- Proporcionar información precisa y práctica sobre el reciclaje de objetos específicos.
- Aumentar la conciencia sobre la importancia del reciclaje y la correcta separación de residuos.
- Hacer el aprendizaje sobre reciclaje más dinámico y memorable.

Sociales:

- Fomentar hábitos de reciclaje responsables en la comunidad.
- Contribuir, aunque sea a pequeña escala, a la reducción de residuos en vertederos y la promoción de una economía circular.

Tecnológicos:

- Integrar con éxito la detección de objetos (YOLO) con la generación de lenguaje natural (ChatGPT) y la visualización 3D (Three.js).
- Implementar una comunicación eficiente mediante WebSockets.
- Demostrar la aplicación práctica de técnicas de computación visual e IA para resolver un problema cotidiano.

Levantamiento de Requerimientos**Requerimientos Funcionales (RF):**

RF01: El sistema deberá permitir al usuario subir una imagen de residuos desde su dispositivo.

RF02: El sistema deberá procesar la imagen subida utilizando un modelo YOLO para detectar objetos de basura.

RF03: El sistema deberá permitir al usuario activar la cámara de su dispositivo para la detección en tiempo real.

RF04: El sistema deberá procesar los frames de la cámara en tiempo real usando YOLO para detectar objetos de basura.

RF05: El sistema deberá enviar los objetos detectados (clases) al backend.

RF06: El backend deberá consultar a la API de ChatGPT para obtener sugerencias de reciclaje basadas en los objetos detectados.

RF07: El sistema deberá mostrar al usuario los objetos detectados.

RF08: El sistema deberá mostrar al usuario las sugerencias de reciclaje proporcionadas por ChatGPT.

RF09: El sistema deberá renderizar modelos 3D representativos de los objetos detectados en una escena de Three.js.

RF10: El usuario deberá poder navegar en la escena 3D (rotar, zoom).

RF12: El sistema deberá presentar una landing page con información básica sobre el proyecto y el reciclaje.

Requerimientos No Funcionales (RNF):

RNF01 (Rendimiento): La detección de objetos a partir de una imagen subida no debería tardar más de 7-10 segundos (incluyendo YOLO y ChatGPT).

RNF02 (Rendimiento): La detección en tiempo real debería procesar al menos 2-3 frames por segundo (considerando la cadena completa: captura, envío, YOLO, envío de vuelta). La llamada a ChatGPT en tiempo real será optimizada (ej. solo con cambios significativos o acumulados) para no degradar excesivamente la experiencia.

RNF03 (Usabilidad): La interfaz de usuario debe ser intuitiva y fácil de usar, con instrucciones claras.

RNF04 (Compatibilidad): El frontend deberá ser compatible con las últimas versiones de navegadores web modernos (Chrome, Firefox, Edge, Safari).

RNF05 (Disponibilidad): El sistema debe estar disponible para su evaluación (desplegado o fácilmente ejecutable localmente).

RNF06 (Precisión IA): El modelo YOLO debe tener una precisión razonable en la identificación de las categorías de basura más comunes para las que fue entrenado/ajustado.

Restricciones:

TEC01: El frontend debe utilizar Three.js.

TEC02: El backend (si se usa para IA) debe ser en Python (Flask).

TEC03: Se debe utilizar un modelo YOLO para la detección de objetos.

TEC04: Se debe utilizar la API de ChatGPT para generar sugerencias de reciclaje.

COST01: El uso de la API de OpenAI (ChatGPT) tiene costos asociados, por lo que las llamadas deben ser gestionadas eficientemente, especialmente en el modo de tiempo real.

DAT01: Se dependerá de modelos 3D preexistentes o simples para los objetos, o de la capacidad de crearlos.

PRIV01: El usuario debe otorgar permisos explícitos para el uso de la cámara. No se almacenarán imágenes ni videos de los usuarios más allá del procesamiento temporal necesario.

Supuestos de Diseño:

- **USU01:** Los usuarios tienen acceso a un dispositivo con navegador web moderno y conexión a internet.
- **USU02:** Los usuarios que utilicen la función de tiempo real disponen de una cámara web o imágenes en el dispositivo.
- **IA01:** El modelo YOLO estará pre-entrenado o ajustado para reconocer un conjunto definido de objetos de basura comunes (ej. botellas de plástico, latas, papel, cartón, vidrio).
- **API01:** La API de OpenAI (ChatGPT) estará disponible y operativa durante el desarrollo y demostración.
- **MOD3D01:** Se dispondrá de modelos 3D (.glb/.gltf) para los principales tipos de residuos a detectar.
- **Usuarios objetivo:** Jóvenes y adultos interesados en el reciclaje o con poca experiencia en clasificación de residuos.
- **Necesidades:** Guía educativa personalizada y visual, entorno lúdico e informativo que no requiere conocimientos previos.

Requerimientos Técnicos y Tecnológicos

Lenguajes de Programación:

JavaScript: Utilizado en el frontend para la visualización interactiva en 3D con Three.js, manejo de eventos, renderizado y conexión con el backend.

Python: Utilizado en el backend para procesamiento visual con YOLO, generación de texto con ChatGPT y gestión de comunicación con el frontend.

HTML/CSS: Para la estructura de la landing page y estilos complementarios a Three.js.

Frameworks o Bibliotecas a Utilizar:

Frontend:

Three.js: Motor de visualización 3D en el navegador. Permite renderizar escenas, objetos y materiales en WebGL.

OrbitControls (Three.js): Para navegación 3D interactiva con mouse o touch.

GLTFLoader: Para cargar modelos .glb o .gltf si se usan representaciones realistas de objetos reciclables.

Backend:

Flask: Microframework web para crear la API y manejar las peticiones.

Flask-SocketIO: Para la comunicación bidireccional en tiempo real (WebSockets).

Librería de YOLO: ultralytics para YOLOv5/YOLOv8, o la librería oficial del modelo YOLO elegido.

OpenAI Python Library: Para interactuar con la API de ChatGPT.

IA:

YOLO: Modelo de detección de objetos (versión a definir, ej., YOLOv5, YOLOv8). Apartir de pesos pre-entrenados.

ChatGPT: Modelo de lenguaje de OpenAI (a través de su API, ej. gpt-3.5-turbo o gpt-4 si el presupuesto lo permite).

Protocolos de Comunicación

REST (HTTP POST): Para la carga inicial de la página, la subida de imágenes estáticas y las respuestas del backend para este flujo.

WebSockets (WS/WSS): Para la transmisión de frames de la cámara del usuario al backend y el envío de resultados de detección y sugerencias en tiempo real desde el backend al frontend.

Propuesta de Arquitectura del Sistema:

Arquitectura Cliente-Servidor:

Cliente (Frontend - Navegador Web):

- Responsable de la interfaz de usuario (HTML, CSS).
- Manejo de la lógica de Three.js para la escena 3D y la interacción.
- Captura de imagen (subida o cámara).
- Envío de imagen/frames al backend vía HTTP o WebSockets.
- Recepción de resultados del backend y actualización dinámica de la UI y la escena 3D.

Servidor (Backend - Python/Flask):**API Endpoints (Flask):**

Recibe la imagen, la pasa a YOLO, consulta ChatGPT, devuelve resultados (detecciones, modelos 3D a usar, sugerencias).

Manejo de WebSockets (Flask-SocketIO):

Establece conexión con el cliente. Recibe un frame de video, lo pasa a YOLO, (opcionalmente y optimizado) consulta ChatGPT, emite resultados de vuelta.

Módulo de IA:

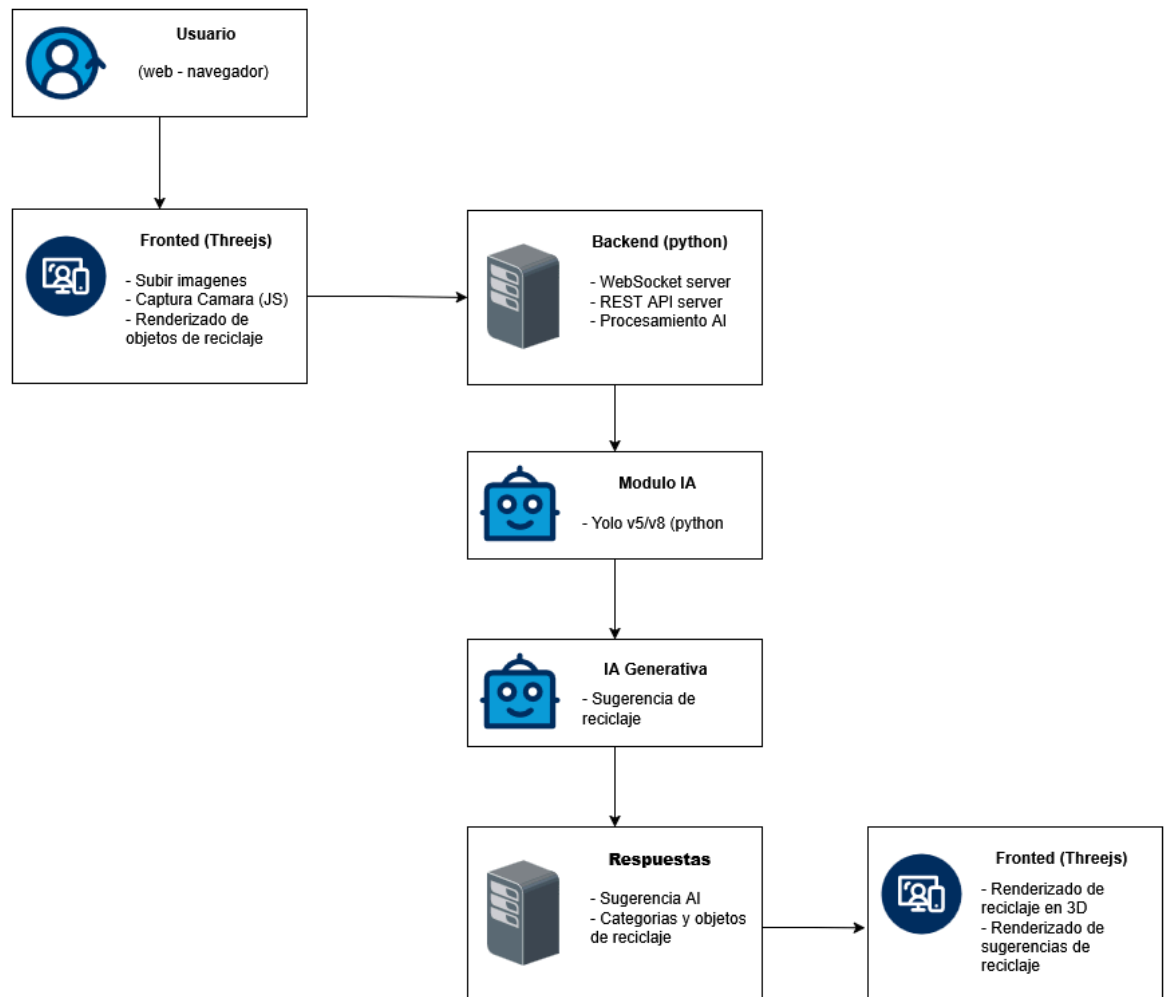
Integración con el modelo YOLO para realizar inferencias.

Cliente para la API de ChatGPT para generar texto.

Diagramas y Arquitectura

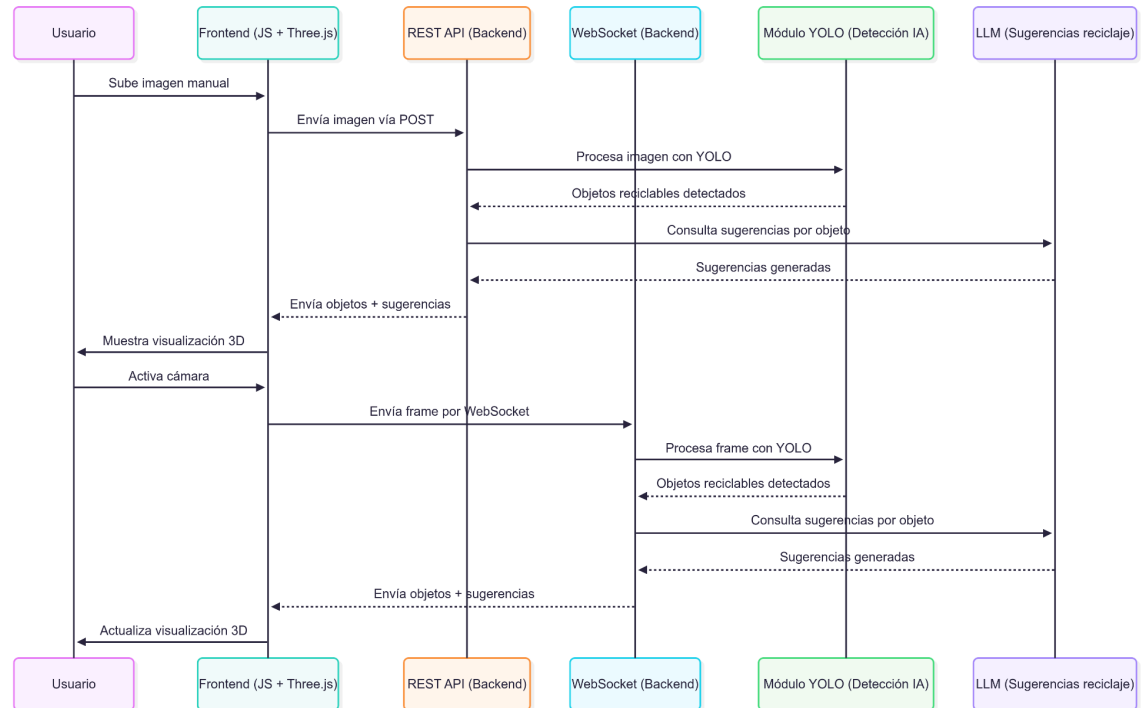
1. Diagrama general del sistema

A continuación, se presenta un diagrama general de la arquitectura del sistema, donde se pueden observar todos los componentes y la forma en que interactúan entre sí. El cliente estará representado por un navegador que ejecuta una aplicación desarrollada en Three.js. Por su parte, el backend estará construido en Python y se encargará de gestionar los distintos servicios, tanto a través de peticiones REST como mediante WebSockets, para habilitar la funcionalidad en tiempo real.



2. Diagrama de interacción de usuario

Este diagrama muestra paso a paso cómo fluye la interacción entre el usuario, la interfaz web y los servicios de backend (REST, websockets) junto con los módulos de inteligencia artificial.



3. Módulos previstos

Módulo	Descripción
Frontend (Three.js + JS)	Subida de imágenes, captura de cámara (JS), comunicación WebSocket, renderizado 3D.
Servidor WebSocket	Comunicación bidireccional continua entre frontend y backend.
Backend (FastAPI/Flask)	Orquestación entre recepción de imágenes, detección IA y generación de texto.
Detección (YOLOv5/8)	Identificación de objetos reciclables en imágenes.
IA generativa (LLM)	Generación de sugerencias de reciclaje con base en los objetos detectados.
Render 3D dinámico	Visualización interactiva de los objetos reciclables detectados.

4. Principales dependencias y riesgos técnicos

Riesgo / Dependencia	Descripción
WebSockets	Necesario mantener conexión estable para envío de imágenes en tiempo real.
Carga de procesamiento de frames	Procesar múltiples imágenes en tiempo real puede generar latencia.
YOLO v5/8	Debe estar bien entrenado para identificar correctamente objetos reciclables.
Generación LLM	Dependencia de una API externa para sugerencias, requiere conexión y manejo de errores.
Three.js	Renderizado debe ser optimizado para múltiples objetos y buena experiencia UX.
Captura de cámara desde JS	Uso de la api del navegador para poder acceder a la cámara, esto puede requerir manejo de permisos del navegador.
Privacidad del usuario	Las imágenes deben ser procesadas temporalmente y nunca almacenadas sin consentimiento.

Repositorios y Recursos

Repositorio

Todo el desarrollo del proyecto puede explorarse en nuestro repositorio de GitHub.

<https://github.com/SergioR-02/computacion-visual-grupo/tree/main/proyecto>

Recursos

YOLOv5 / YOLOv8 – Ultralytics

Framework de detección de objetos de alto rendimiento, ideal para ejecutar en el backend con Python: <https://github.com/ultralytics/yolov5>

OpenAI API (ChatGPT)

Utilizada para generar recomendaciones personalizadas de reciclaje a partir de objetos detectado: <https://platform.openai.com/docs>

Herramientas de Desarrollo

Three.js

Biblioteca para visualización 3D en el navegador basada en WebGL:

<https://threejs.org/>

Flask + Flask-SocketIO

Microframework para desarrollo de backend en Python y manejo de WebSockets:

<https://flask-socketio.readthedocs.io/>

Recursos de Soporte y Referencia

OpenCV (para preprocesamiento de imágenes) <https://opencv.org/>

GLTF Model Repository (Modelos 3D gratuitos para Three.js)

<https://sketchfab.com>

<https://poly.pizza/>

<https://github.com/KhronosGroup/glTF-Sample-Models>

Mockups o Prototipos

El diseño de los mockups de EcoScan 3D se basa en principios de accesibilidad, modernidad y orientación educativa, con una estética pulida que combina tecnología y cercanía al usuario. La interfaz tiene como objetivo principal facilitar la comprensión del reciclaje mediante herramientas interactivas, visuales y automatizadas.

Vista de Inicio – Landing Page

Esta vista representa la entrada a la aplicación. Su función es atraer al usuario, comunicar el propósito del proyecto y motivarlo a explorar la herramienta. El diseño contempla:

- Un **titular llamativo y conciso** que capta la atención de inmediato.
- Una **breve descripción** que resalta el uso de inteligencia artificial y modelos 3D para facilitar el aprendizaje.
- Un **botón de llamado a la acción (CTA)** grande y visible, con textos como “Empezar a Reciclar” o “Probar la App”.
- Una **animación de fondo sutil**, relacionada con el reciclaje o elementos 3D, para reforzar el carácter tecnológico sin distraer.





Vista Principal – Dashboard Interactivo

Esta sección representa el núcleo funcional de la plataforma. Se ha estructurado como un dashboard dividido en cuatro módulos, diseñados para integrarse de forma lógica y no sobrecargar visualmente al usuario.

Módulo 1: Detección en Tiempo Real (Cámara)

Permite usar la cámara del dispositivo para identificar residuos automáticamente. Se muestran los objetos detectados superpuestos en el video o en una lista contigua, junto a controles simples para iniciar o detener la cámara.

Módulo 2: Análisis de Imagen Estática

Ofrece la alternativa de subir una imagen para su análisis. El usuario puede arrastrar o seleccionar una foto, y verá los resultados con etiquetas y recuadros sobre los elementos detectados.

Módulo 3: Asistente de Reciclaje

Después de una detección, este módulo se activa automáticamente para mostrar sugerencias prácticas sobre cómo reciclar cada objeto, utilizando información generada por ChatGPT. Las recomendaciones son breves, claras y enfocadas en acciones concretas.

Módulo 4: Visualizador 3D

Este visor presenta un modelo 3D del material detectado, como plástico, vidrio o cartón, permitiendo rotación y zoom. Esto refuerza el componente visual y pedagógico de la aplicación, haciendo el aprendizaje más memorable.

